

Threaded Prime Number Search

Gideon Chua

Flow for All Task Division Schemes

- Precompute a number of divisors from 2 to $\sqrt{\text{range}}$
- Divide some number n , where $2 \leq n \leq \text{range}$, using the divisors
- $n \bmod \text{divisor}$
- If none of the divisors are able to divide the number cleanly, then the number is prime, and can be stored somewhere
- Otherwise, skip the number.

Precomputing Divisors

```
std::vector<int> divisors;  
for(int i = 2; i <= range / i; i++){  
    divisors.push_back(i);  
}
```

Prime Checker (Straight Division)

```
std::vector<int> result;
for (int i = range_begin; i <= range_end; i++){
    for (const auto& div : divisors){
        if(i % div == 0 && i != div){
            break;
        }
        else if(div == divisors.back()){
            result.push_back(i);
        }
    }
}
results[id] = std::move(result);
```

Task Division (Straight Division)

```
int segment = threads > 0 ? (range - 1) / threads : 0;
for(int i = 2; threads > 0 && i <= range; i += segment + 1){
    if(i + segment >= range){
        thread_list.emplace_back(primechecker, i to range..);
    }
    else{
        thread_list.emplace_back(primechecker, i to segment..);
    }
}
for(auto& th : thread_list) th.join();
```

Prime Checker (Divisibility Threads)

```
void primecheck_job_scheme2(int id, int number,  
    const std::vector<int>& divisors, std::vector<int> &results){  
    for (const auto& div : divisors){  
        if(number % div == 0 && number != div){  
            break;  
        }  
        else if(div == divisors.back()){  
            results[id] = number;  
        }  
    }  
}
```

One thread tests one number. The search itself stays linear.

Task Division (Divisibility Threads)

```
for(int i = 2; threads > 0 && i <= range; i += threads){  
    id = 0;  
    for (int j = i; j < i + threads && j <= range; j++){  
        thread_list.emplace_back(primecheck_job_scheme2,  
                                id++, j, divisors, results);  
    }  
    for (auto& th : thread_list) th.join();    // wait for the batch  
    thread_list.clear();  
}
```

Every batch of threads numbers ends in a join.

Zero Threads

```
// threads 0: just run on the main thread
if (threads == 0)
    primecheck_job_scheme1(0, 2, range, divisors, results);
```

threads 0 in config.txt gives a no-threading baseline to compare against.

Printing Variants

Print immediately. Thread id and timestamp, printed the moment a prime is found.

```
std::ostream(std::cout) << "(" << std::format("{:%H:%M:%S}", ms) << ")"  
    << " Thread ID: " << id << " found prime: " << i << "\n";
```

Print at end. Threads only store results. Join, merge, sort, then print once.

```
std::sort(all.begin(), all.end());  
for (const auto& r : all) std::cout << r << " ";
```

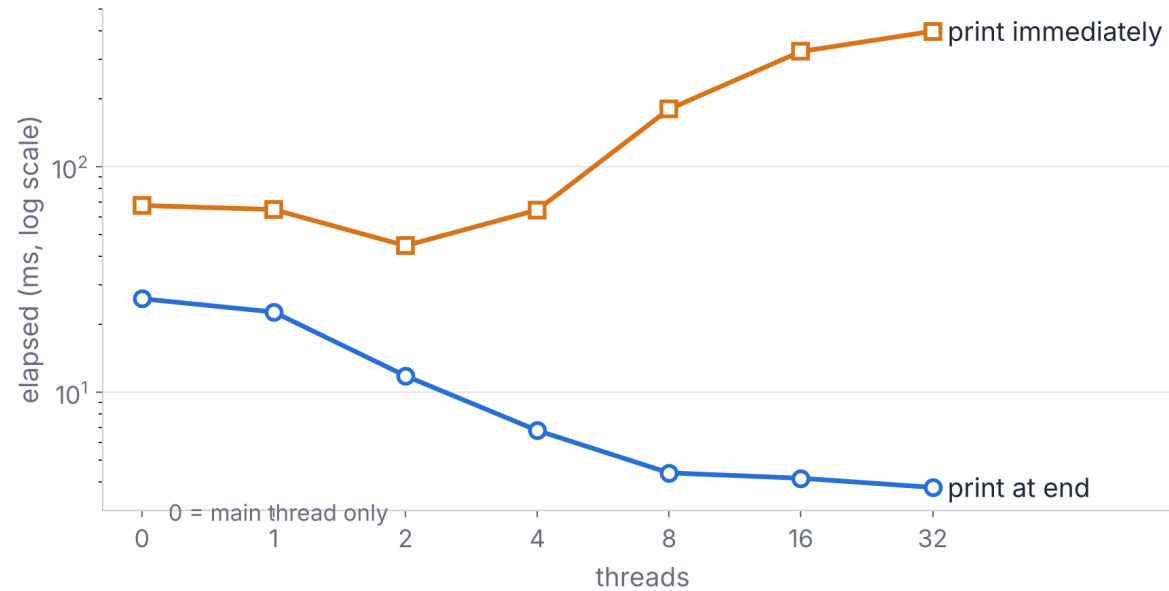
Timing

- Start: and End: wall-clock timestamps are printed on every run.
- Elapsed time uses `std::chrono::steady_clock`, which does not jump if the system clock changes.
- The end timestamp is taken right after the join, before any printing in the print-at-end variants.

Benchmark Setup

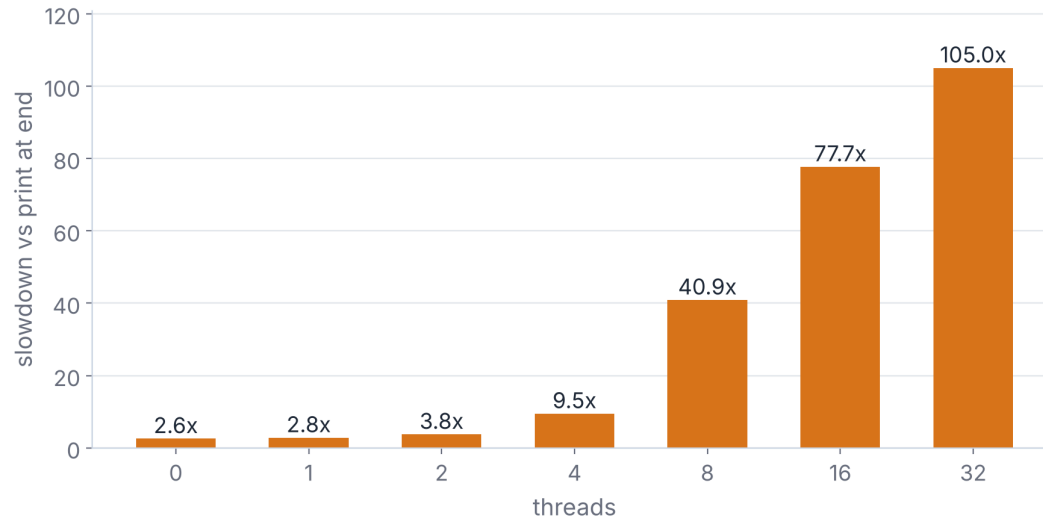
- Range: 100,000
- Threads: 0 (main only), 1, 2, 4, 8, 16, 32
- 3 runs per setting, averaged
- Program output sent to `/dev/null`, so terminal rendering is not measured
- Same machine, release builds

Straight Division



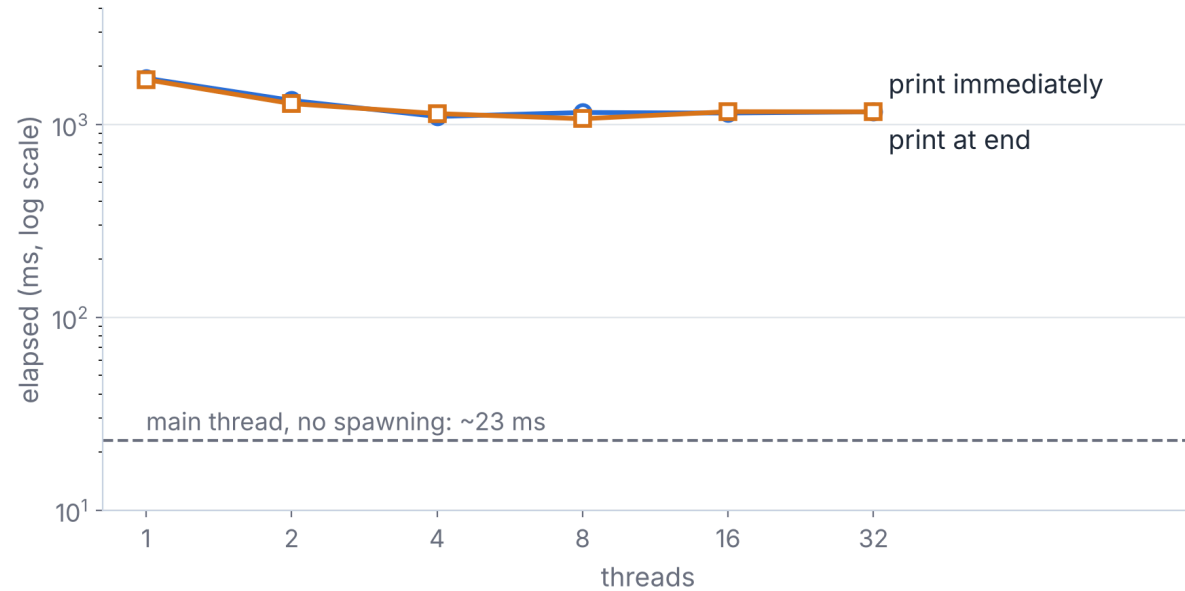
Print at end: 23 ms on one thread, 12 ms on two, 4 ms by 8-16 threads. Gains flatten past 8.

The Cost of Printing Immediately



Printing immediately is 2.6x slower on the main thread and about 100x slower at 32 threads (398 vs 4 ms). It is fastest at 2 threads (45 ms), then gets worse with every thread added, since threads wait on the shared output lock instead of working.

Divisibility Threads



1,700 ms on one thread, 1,100 ms from 4 threads up. About 50x slower than running on the main thread. Printing mode barely matters here.

Interleaved Output

Lines from different threads arrive out of order, so the output is not sorted. `osyncstream` keeps each line whole, but not in order.

Takeaways

- **Threads help only when the work per thread is large enough.**
Straight division scales until the chunks get small.
- **Shared output is a bottleneck.** A lock around cout makes extra threads slower.
- **Joining is a cost.** Divisibility threads pay for thread creation and a join barrier on every batch.
- **Output order is not guaranteed** when threads print as they go.
- **Divide coarsely.** Splitting the range beats splitting each number.